

IA Engineer Core Track: LLM Engineering, RAG, QLoRA, Agents

[18-06-2025-]



1.Seccion 1: Week 1 – Build Your First LLM Product: Exploring Top Models

1.1 Dia 1: Ollama: Ejecutar Modelos de IA en local

1.1.1 ¿Qué es Ollama?

Ollama es un programa gratuito y de código abierto que permite **descargar y correr modelos de lenguaje (LLMs) directamente en la propia computadora**, sin necesidad de interne una vez descargado el modelo, sin pagar por uso, y sin enviar los mensajes a ningún servidor externo.

La comunidad lo llama el **"Docker de los modelos lingüísticos"**: abstrae toda la parte técnica (drivers, formatos de archivo, dependencias) y reduce todo a un simple comando de terminal.

¿Qué hace exactamente?

Función	Qué significa
Descarga modelos	<code>ollama pull <modelo></code> baja el modelo desde un repositorio público
Los corre localmente	Carga el modelo en la RAM/GPU y abre un chat interactivo
Gestiona memoria	Carga el modelo cuando se necesita y lo libera al cambiar de modelo
Expone una API local	Levanta un servidor en el puerto 11434, compatible con la API de OpenAI

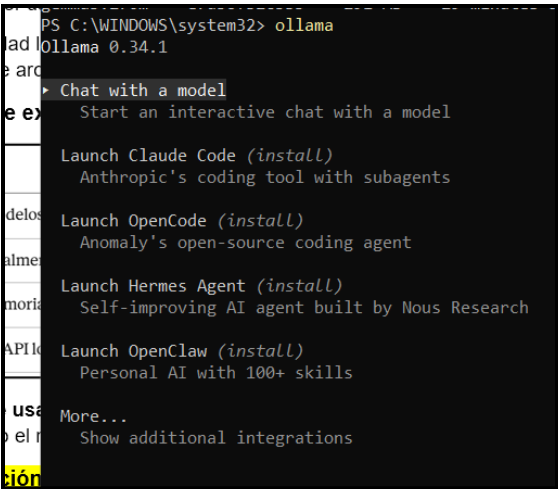
Por que se usa: privacidad total (nada sale de la PC), sin costo por token, funciona offlina una vez descargado el modelo, y sin pelear con entornos de Python ni drivers de GPU.

1.1.2 Instalación y primeros pasos

El proceso es el mismo en Mac, Windows y Linux — solo cambia la terminal que se usa:

	Mac	Windows
Terminal	Terminal (zsh)	PowerShell
Cómo abrirla	Aplicaciones → Utilidades → Terminal	Menú Inicio → escribir "PowerShell"
Comandos	Idénticos	Idénticos

1. Descargar Ollama desde ollama.com e instalarlo (arrastrar a Aplicaciones en Mac, instalador normal en Windows).
2. Verificar que está instalado, escribiendo en la terminal:



Debería mostrar la lista de comandos disponibles (**serve**, **create**, **run**, **pull**, **list**, **etc.**). Nota: en versiones recientes de Ollama puede aparecer un menú interactivo distinto al de versiones antiguas — se sale con la tecla Esc. 3. Confirmar que el servidor ya está corriendo:

```
PS C:\WINDOWS\system32> ollama serve
Error: listen tcp 127.0.0.1:11434: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
PS C:\WINDOWS\system32>
```

Si tira un error de "dirección ya en uso" (*address already in use*), es buena señal: significa que Ollama ya está activo en segundo plano.

1.1.3 Comandos básicos de Ollama

Comando	Qué hace
<code>ollama run <modelo></code>	Descarga el modelo (si no está) y abre un chat interactivo con él
<code>ollama pull <modelo></code>	Solo descarga el modelo, sin abrir el chat
<code>ollama list</code>	Lista todos los modelos ya descargados en el disco
<code>ollama show <modelo></code>	Muestra detalles técnicos del modelo (parámetros, arquitectura, etc.)
<code>ollama rm <modelo></code>	Borra un modelo del disco
<code>Ctrl + D</code>	Sale de la conversación activa con un modelo

1.1.4 Modelos probados en clase

El numero después de los dos puntos en el nombre del modelo indica la cantidad de parámetros (**m** = millones, **b** = mil millones en español)

Modelo	Parámetros	Tamaño disco	Resultado en la prueba
<code>gemma3:270m</code>	270.000.000	292 MB	Respuestas simples, corrió sin problema
<code>phi3</code> (mini)	3.800.000.000	2.2 GB	Coherente, dio datos curiosos correctos (Torre Eiffel, Joey Chestnut)
<code>gpt-oss:20b</code>	20.900.000.000	13 GB	Descargó y corrió, pero generaba lentitud / se sentía al límite de la RAM — se borró luego con <code>ollama rm</code>

Nota sobre gpt-oss: es un modelo de razonamiento (*reasoning model*). Antes de responder, muestra un bloque "Thinking..." donde razona internamente qué está preguntando el usuario y cómo responder, antes de dar la respuesta final. Misma idea que o1 de OpenAI o el modo "extended thinking" de Claude.

1.1.5 Por que importa la RAM

A diferencia de un video que se puede ir leyendo de a poco, un modelo de lenguaje necesita **TODOS** sus parámetros cargados en memoria al mismo tiempo, porque cada palabra generada requiere cálculos que involucren todos los parámetros.

Regla aproximada: Con la comprensión (quantizacion) que usa Ollama por defecto, cada mil millones de parámetros ocupa aproximadamente 0.5-0.6 GB en memoria.

Ejemplo con gpt-oss:20b → $20.9 \times 0.6 \approx \mathbf{14\ GB}$ solo para cargar el modelo, sin contar el sistema operativo ni otros programas.

Caso de esta PC: 15.8 GB de RAM total. Con eso, gpt-oss:20b queda muy al límite y puede trabarse; phi3 (3.8B) y gemma3:4b son opciones estables sin arriesgar rendimiento.

1.1.6 Que modelo usar según el hardware (vistos en clase)

Modelo	RAM aprox.	Recomendación
<code>gemma3:270m</code>	< 1 GB	Muy liviano, respuestas básicas
<code>phi3</code> (3.8b)	~4-5 GB	Recomendado — estable con 16 GB de RAM
<code>gemma3:4b</code>	~5-6 GB	Alternativa estable, soporta imágenes
<code>gemma3:12b</code>	~10-12 GB	Al límite con 16 GB, no recomendado
<code>gpt-oss:20b</code>	~14-16 GB	No recomendado con 16 GB, riesgo de crasheo

1.1.7 Los modelos más potentes disponibles hoy (2026)

Ojo con algo importante: **no todos los modelos que aparecen en el catálogo de Ollama corren en la propia computadora**. Hay dos tipos:

- **Locales:** se descargan con ollama pull/ollama run y corren 100% en el hardware propio (lo que se estuvo usando toda la clase: Gemma, Phi-3, gpt-oss).
- **:cloud:** llevan ese sufijo en el nombre (ej. **kimi-k2.6:cloud**). Ollama los ejecuta en sus propios servidores — el comando se escribe igual, pero en realidad se le pega por internet a una máquina de Ollama, no a la propia PC. Suelen requerir cuenta y, para los más grandes, un plan pago.

Modelo	Tipo	Parametros totales	Conte xto	RAM/ VRAM real necesaria	Plan de pago necesar io	Caso de uso más común	Ejemplo concreto de uso
Kimi K2.6	:cloud únicamente	1.000.000.00 0.000 (1T)	256K	~350 GB de RAM/VRAM — imposible en una PC de escritorio o laptop; ni siquiera existe versión descarga ble, solo corre en servidore s de Ollama	Pro (\$20/me s) o Max (\$100/me s) — es de uso "alto"	Agentes autónomos complejos: coordinar hasta 300 sub-agentes en paralelo, tareas de +4.000 llamadas a herramientas	Un equipo de desarrollo le pide "generará un sistema completo de facturación con backend, frontend y tests" y el modelo despliega decenas de sub-agentes que trabajan en paralelo durante horas hasta entregar más de 100 archivos
DeepSeek-V4 Pro	:cloud únicamente	Cientos de miles de millones (MoE)	1M	Decenas a cientos de GB — solo viable en servidore s con múltiples GPUs de datacent er	Max (\$100/me s) — es de uso "extra alto"	Razonamiento de frontera y problemas de programación difíciles	Depurar un bug intermitent e en un sistema distribuido complejo, donde el modelo tiene que razonar paso a paso sobre

							condicione s de carrera antes de proponer una solución
Llama 4 Maverick	Local (con hardware serio)	400.000.000. 000 (MoE)	1M	Decenas de GB de VRAM — requiere una o varias GPUs de gama alta (ej. workstati on con 48-80 GB de VRAM)	No aplica (es local, pero necesita GPU propia muy potente)	Documentos o codebases enteros gracias a su contexto de hasta 1 millón de tokens	Cargarle un repositorio entero de una app grande (miles de archivos) en una sola conversaci ón y pedirle que explique cómo se conecta el módulo de pagos con el de usuarios
GLM-5.2 / Mistral Large 3	:cloud principalm ente	100.000.000. 000+	128K+	40-80 GB de VRAM — GPU de datacent er (ej. A100/H1 00) o varias GPUs de consumo en paralelo	Pro o Max según el modelo (uso "alto")	Ingeniería de software en repositorios grandes y tareas multilingües/multim odales	Una empresa con equipos en distintos países le pide analizar documenta ción técnica en inglés, chino y español y generar un resumen unificado con capturas de pantalla incluidas
Qwen3.6: 27b	Local	27.000.000.0 00	256K	~17 GB de RAM/VR AM — una GPU de 24 GB (ej. RTX 4090) o una Mac	No aplica (gratis, corre local)	Programación seria, razonamiento y RAG — el más recomendado para un solo modelo local	Un desarrollad or freelance lo usa como asistente de código diario en su PC

				con 32 GB de memoria unificada			personal, sin pagar API, para refactorizar funciones y escribir tests
Llama 3.3:70b	Local (con GPU potente)	70.000.000.000	128K	~43 GB de VRAM — GPU de gama alta o varias GPUs de consumo combinadas	No aplica (gratis, corre local)	Chat general y RAG (recuperación de documentos) como "caballo de batalla"	Un chatbot interno de una empresa que responde preguntas de empleados consultando manuales internos, corriendo en un servidor propio por temas de privacidad
Gemma 3:27b	Local	27.000.000.000	128K	~17 GB de RAM/VRAM — una GPU de 24 GB o una Mac con 32 GB de memoria unificada	No aplica (gratis, corre local)	Escritura con matices, documentos largos e instrucciones complejas	Redactar y revisar un informe extenso (decenas de páginas) manteniendo un tono consistente de principio a fin
gpt-oss:120b	Local (con GPU potente)	116.830.000.000 (MoE)	128K	~65-80 GB de VRAM — GPU de datacenter de 80 GB (ej. A100/H100)	No aplica (gratis, corre local)	Razonamiento con capacidad cercana a modelos cerrados grandes, en hardware propio de gama alta	Una universidad o laboratorio con un servidor propio lo usa para investigación, evitando mandar datos sensibles a una API externa
gpt-oss:20b (el	Local	20.910.000.000 (MoE)	128K	~16 GB de RAM — una	No aplica (gratis,	Chat general y primeras pruebas de razonamiento	Justo lo que se hizo en esta

probado en clase)	PC doméstica con 16 GB, aunque al límite (como se comprobó en clase)	corre local)	en una PC doméstica	clase: instalarlo en una laptop de 16 GB de RAM y probar cómo "piensa" antes de responder, sin depender de internet
--------------------------	--	--------------	---------------------	---

Sobre el plan de pago (Ollama Cloud): solo hace falta pagar cuando se quiere usar un modelo con sufijo :cloud (Ollama lo ejecuta en sus servidores). Los modelos **locales** (los que se descargan y corren en la propia PC, como todos los usados en clase) son siempre gratis — lo único que "cuesta" es el hardware propio.

Plan	Precio	Qué incluye
Free	\$0	1 modelo cloud a la vez, uso liviano
Pro	\$20/mes (o \$200/año)	3 modelos cloud a la vez, 50 veces más uso que Free
Max	\$100/mes	10 modelos cloud a la vez, 5 veces más uso que Pro

Conclusión práctica: de los modelos "top" que existen hoy, los más grandes de todos (Kimi K2.6, DeepSeek-V4 Pro) **no se pueden correr localmente ni con la mejor PC de escritorio** — solo están disponibles vía :cloud, o sea, pagándole a Ollama para que lo corra en su infraestructura. El techo real de lo que se puede *descargar y ejecutar en una máquina propia* está en modelos como gpt-oss:120b o llama3.3:70b (necesitan GPUs de gama alta con 40-80 GB de VRAM), y bajando a hardware doméstico normal (16 GB de RAM, como en esta PC), el techo realista sigue siendo gpt-oss:20b o modelos densos de 7B-12B como gemma3:12b o phi3.

Dato interesante: varios de los modelos más grandes (Kimi K2.6, gpt-oss, Llama 4) usan arquitectura **MoE (Mixture of Experts)** — tienen billones de parámetros "guardados", pero por cada palabra que generan solo activan una fracción pequeña de esos parámetros (los "expertos" relevantes para esa tarea). Por eso el uso real de memoria depende más de los parámetros *activos* que de los *totales*.

1.1.8 Como borrar un modelo o desinstalar Ollama

Borrar solo un modelo (recomendado si el problema es un modelo puntual, ej. gpt-oss):

```
>>>
PS C:\WINDOWS\system32> ollama rm gpt-oss
deleted 'gpt-oss'
```

Ver qué modelos quedan instalados:

```
deleted 'gpt-oss'
PS C:\WINDOWS\system32> ollama list
```

NAME	ID	SIZE	MODIFIED
phi3:latest	4f2222927938	2.2 GB	16 minutes ago
gemma3:270m	e7d36fb2c3b3	291 MB	29 minutes ago

Desinstalar Ollama por completo (Windows):

1. Inicio → Configuración → Aplicaciones → Aplicaciones instaladas → buscar "Ollama" → Desinstalar.
2. **Importante:** desinstalar el programa NO borra los modelos descargados. Hay que borrar manualmente la carpeta %USERPROFILE%\ollama para recuperar el espacio en disco.

1.2 Día 2: Roadmap del curso: LLM Engineering (8 semanas)



El recorrido semana a semana

Semana	Tema
1	Fundamentos y la Chat Completions API (<i>estamos acá</i>)
2	Modelos frontera (frontier models) con APIs, interfaces de usuario y multimodalidad
3	Código abierto con Hugging Face
4	Selección de LLMs y generación de código
5	RAG y Question Answering — creando un "experto"
6	Fine-tuning de un modelo frontera
7	Fine-tuning de un modelo de código abierto
8	El final — IA agéntica

Al completar la semana 8, el objetivo declarado del profesor es que el alumno pueda decir "ahora soy ingeniero LLM" — llegar a ese hito de dominio del tema.

1.2.1 La lógica del curso: construye sobre lo anterior

El profesor remarca que el curso "se construye y se construye y se construye" — no son 8 temas sueltos, sino una progresión:

1. **Primero, panorama de modelos** — variedad de LLMs, tanto de código abierto como cerrado (lo que se estuvo haciendo con Ollama: Gemma, Phi-3, gpt-oss).
2. **Después, las herramientas** – kits y librerías para trabajar con esos LLMs.
3. **Luego, las técnicas** – formas de sacarles el máximo rendimiento a los modelos
4. **Y todo el tiempo, proyectos reales** — el profesor insiste en que la mejor forma de aprender es construyendo proyectos comerciales grandes, no solo teoría.

1.2.2 Las 3 técnicas centrales que se van a cubrir

Técnica	Para que sirve
RAG (Retrieval-Augmented Generation)	Darle a un modelo acceso a información externa (documentos propios) para que responda con datos que no tenía en su entrenamiento
Fine-tuning	Ajustar/entrenar un modelo existente con datos propios para especializarlo en una tarea
Agents (agentes)	Que un modelo pueda actuar de forma autónoma, tomando decisiones y usando herramientas por su cuenta

1.2.3 Las herramientas/frameworks que se van a usar

Herramienta	Para que
Hugging Face	Plataforma central de modelos de código abierto. Es una plataforma colaborativa de código abierto considerada el “Github de la IA”, donde millones de desarrolladores comparten modelos, datos y aplicaciones de aprendizaje automático.
Gradio	Crear interfaces de usuario rápidas para probar modelos
LangChain	Framework para encadenar llamadas a LLMs y construir aplicaciones
W&B (Weights & Biases)	Seguimiento y registro de experimentos de entrenamiento
Modal	Infraestructura en la nube para correr cargas de trabajo pesadas (ej. entrenamiento con GPU)

1.2.4 Los modelos que se van a explorar a lo largo del curso

Modelos cerrados (frontera):

- GPT-5 (OpenAI)
- Claude (Anthropic)
- Gemini (Google)

Modelos de código abierto:

- Llama (Meta)
- Phi (Microsoft) — ya probado en Ollama
- Qwen (Alibaba)

1.2.5 Tipos de proyectos que vas a construir

El profesor menciona explícitamente:

- Proyectos **multimodales** (texto, imagen, etc. Combinados)
- Aplicaciones **comerciales de consumo**
- Algo más técnico: **portar código de un lenguaje a otro**
- Proyectos que involucran **vectores** (embeddings, bases de datos vectoriales — clave para RAG)
- **Entrenamiento con GPUs** a mayor escala
- Un **gran proyecto comercial final**, al cierre de las 8 semanas.

1.2.6 Ejemplo práctico mostrado en el video

Como cierre de la parte de Ollama, el profesor mostró un caso de uso real con gpt-oss corriendo local: le pidió que actuara de **tutor de español para principiantes**. El modelo:

1. Mostró su bloque de "pensamiento" (Thinking...) antes de responder
2. Se presentó en español como tutor, con un tono natural y cercano
3. Empezó una conversación guiada paso a paso (saludo → preguntar el nombre → preguntar de dónde es, etc.)

El profesor lo usa como ejemplo de que, incluso con un modelo corriendo 100% local en la propia PC, ya se puede obtener valor real — comparable a funcionalidades que ofrece una app comercial como Duolingo.